

DoctorStrange — Documentación de Cambios

Correcciones matemáticas y mejoras de accesibilidad

Fecha: 23 de mayo de 2026 · Rama: vectores

Resumen ejecutivo

Se revisó el código completo del proyecto **DoctorStrange**, una herramienta educativa de álgebra lineal desarrollada en Python con CustomTkinter. Se identificaron y corrigieron **5 errores matemáticos/técnicos** y se aplicaron **6 mejoras de accesibilidad** en la interfaz gráfica.

Estructura del proyecto

El proyecto tiene arquitectura Modelo-Vista con dos paquetes principales:

Archivo / Módulo	Descripción
logica/vectores.py	Funciones de análisis de vectores en $\mathbb{R}^n$ (GS, ortonormalización, diagnóstico)
logica/polinomios.py	Análisis de polinomios en $\mathbb{P}^n$ (representados como vectores de coeficientes)
logica/matrices.py	Análisis de conjuntos de matrices con producto de Frobenius
interfaz/tab_vectores.py	Pestaña de la GUI para vectores
interfaz/tab_polinomios.py	Pestaña de la GUI para polinomios
interfaz/tab_matrices.py	Pestaña de la GUI para matrices
interfaz/ventana_principal.py	Ventana principal y gestor de pestañas

Errores corregidos

1. División por cero en Gram-Schmidt (crítico)

Archivo: logica/vectores.py · función `ortogonalizar()` y `ortonormalizar()`

Cuando el usuario ingresaba vectores **linealmente dependientes**, el proceso de Gram-Schmidt producía un vector resultado ≈ 0 . Este vector se agregaba a la lista `ortogonales[]` y en la siguiente iteración se dividía entre su norma al cuadrado ($= 0$), causando **ZeroDivisionError** o valores **NaN/inf**.

Código original (con el bug):

```
for u in ortogonales:
    w = w - (np.dot(w, u) / np.dot(u, u)) * u # crash si np.dot(u,u) == 0
ortogonales.append(w) # agrega el vector cero sin verificar
```

Código corregido:

```
for u in ortogonales:
    norm_sq = np.dot(u, u)
    if not np.isclose(norm_sq, 0): # guard contra división por cero
        w = w - (np.dot(w, u) / norm_sq) * u
```

```
if not np.allclose(w, 0): # solo agrega vectores no-nulos
    ortogonales.append(w)
```

La misma corrección se aplicó en `ortonormalizar()` (guard en la norma antes de dividir) y en las funciones equivalentes de `logica/matrices.py`.

2. Producto interno de Frobenius semánticamente incorrecto (técnico)

Archivo: `logica/matrices.py` · función `producto_interno()`

El código usaba `np.vdot(A, B)`, que aplana las matrices internamente antes de operar. Aunque numéricamente produce el mismo resultado que el producto de Frobenius, la semántica es incorrecta y podría causar errores con distintas formas de array en el futuro.

Cambio:

```
# Antes (semánticamente incorrecto):
return np.vdot(A, B)

# Después (producto de Frobenius explícito):
return np.sum(A * B)
```

3. Función `_sup()` con código muerto y lógica redundante (menor)

Archivo: `logica/polinomios.py` · función `_sup()`

La función que genera superíndices Unicode para las potencias de x tenía una rama `if n <= 3` que era código muerto: los casos $n=2$ y $n=3$ están cubiertos por el diccionario `SUPER` que ya incluye todos los dígitos. Se simplificó la función de 5 líneas a 2.

```
# Antes (con rama redundante):
if n == 0: return ''
if n == 1: return ''
if n <= 3: return {2: '²', 3: '³'}[n] # redundante
return ''.join(SUPER[int(d)] for d in str(n))

# Después:
if n <= 1: return ''
return ''.join(SUPER[int(d)] for d in str(n))
```

4. Gram-Schmidt oculto para conjuntos que no son bases (UX/lógica)

Archivos: los tres archivos de interfaz

El código original tenía un `return` anticipado cuando el conjunto no formaba una base, lo que impedía mostrar el proceso de Gram-Schmidt para conjuntos L.I. no generadores (ej. 2 vectores en $\mathbb{R}^3$). Esta es una situación común en ejercicios educativos.

```
# Antes – si no es base, se corta sin mostrar GS:
if not base:
    return
if not ort:
    # ... mostrar GS ...

# Después – GS siempre visible, con mensaje contextual:
if ort:
```

```
# muestra '✓ El conjunto ya es ortogonal'
else:
    ortogonales = vec.ortogonalizar(matriz)
    if len(ortogonales) == 0:
        # muestra '(todos los vectores son linealmente dependientes)'
    else:
        # muestra los vectores ortogonalizados
```

5. except Exception demasiado amplio (técnico)

Archivos: los tres archivos de interfaz

El bloque de parseo y cálculo estaba envuelto en un único `except Exception`, lo que ocultaba errores de programación y hacía imposible distinguir un error de formato del usuario de un crash interno del código matemático. Se separaron en dos bloques independientes con tipos de excepción específicos.

```
# Después – bloques separados:
try:
    # solo el parseo de la entrada
    vectors = ast.literal_eval(...)
    ...
except (ValueError, SyntaxError):
    # error de formato del usuario

try:
    d = vec.diagnosticar_vectores(matriz)
except Exception as e:
    # error inesperado – muestra el mensaje real
```

Mejoras de accesibilidad e interfaz

Mejora	Detalle
Ctrl+Enter	Atajo de teclado para ejecutar el diagnóstico desde la caja de texto, sin necesidad de usar el ratón
Indicador de atajo	Texto gris "Ctrl+Enter para diagnosticar" bajo el textbox, visible pero discreto.
Contexto del análisis	Primera línea del resultado muestra "Analizando N vectores en ■³" (o equivalente para polinomios)
Rango neutral	El valor de Rango ya no se muestra en verde (el rango es un número, no una propiedad pass/fail)
GS siempre visible	Las secciones de ortogonalización y ortonormalización siempre aparecen, mostrando "✓ ya es ort"
Mensaje para dependientes	Si Gram-Schmidt produce conjunto vacío (todos dependientes), muestra un mensaje explicativo en

Tabla resumen de cambios por archivo

Archivo	Tipo de cambio	Descripción
logica/vectores.py	<i>Bug crítico</i>	Guard división por cero en <code>ortogonalizar()</code> y <code>ortonormalizar()</code>

logica/matrices.py	<i>Bug técnico</i>	Frobenius: np.vdot → np.sum(A*B); mismo guard de cero
logica/polinomios.py	<i>Menor</i>	Simplificación de _sup() (código muerto eliminado)
interfaz/tab_vectores.py	<i>UX + técnico</i>	GS siempre visible, Ctrl+Enter, contexto, Rango neutral, except separado
interfaz/tab_polinomios.py	<i>UX + técnico</i>	Ídem tab_vectores
interfaz/tab_matrices.py	<i>UX + técnico</i>	Ídem tab_vectores